

Claude Code & MCP Tooling — Employee Onboarding Project

Generated by Claude Code

2026-07-20

Claude Code & MCP Tooling — Reference

This document describes the AI-tooling layer wrapped around the Employee Onboarding project: how Claude Code is configured for this repo, which MCP (Model Context Protocol) servers it talks to, the custom skill and scheduled automation that were built on top, and how all of it strings together into the actual Jira → code → Confluence workflow used on this project.

1. `.claude/CLAUDE.md` — the project rulebook

`.claude/CLAUDE.md` is read at the start of every Claude Code session in this repository. It is not application code — it is a standing set of instructions that override Claude's default behavior for this project specifically.

It defines:

- **Tech stack facts:** Java, Maven, Spring Boot, JUnit 5, local SonarQube (Docker, `http://localhost:9000`), Git/GitHub, Jira Cloud, Confluence Cloud.
- **Explicitly disallowed tooling:** Python, pip/virtualenvs, Node.js — Maven is the only build/test/dependency tool permitted.
- **Git rules:** push via `git push` (never embed tokens in the remote URL), never push directly to main, branch naming `feature/<JIRA-ID>-<short-description>`.
- **The Feature Development Workflow** — a fixed 13-step sequence triggered by giving Claude a Jira ID: fetch the ticket → branch → mark In Progress → TDD (failing test first) → implement → Sonar scan and fix issues → commit as `<JIRA-ID>: <summary>` → push → open a PR (never merge) → update README → comment on Jira with the PR link and move it to In Review.
- **Coding standards:** follow existing style, always TDD, PR-only (no merging).
- A rule added mid-project: **always update Confluence once a Jira ticket reaches Done** — this is what motivated the scheduled sync job described in §4.

Every multi-step task performed in this project (e.g. the TC-9 bug fix) follows this file's workflow exactly, rather than any ad hoc process.

2. MCP servers — how Claude reaches Jira, Confluence, and GitHub

MCP (Model Context Protocol) is the mechanism by which Claude Code calls out to external tools instead of being limited to local file access. Available/connected servers can be inspected at any time with:

```
claude mcp list
```

This project's `.mcp.json` registers a single server:

```
{
  "mcpServers": {
    "MCP_DOCKER": {
      "command": "docker",
      "args": ["mcp", "gateway", "run", "--profile", "claude_code_poc"]
    }
  }
}
```

MCP_DOCKER is a **gateway process** — one Docker-run process that itself hosts several independent sub-servers, discovered and enabled dynamically. Three are relevant to this project:

2.1 atlassian — Jira + Confluence

Started out configured for **Jira only** (`jira.url`, `jira.username`). Confluence was added later:

1. `mcp-find` (catalog search) confirmed the same atlassian server also supports Confluence, requiring a `confluence.config` block plus a `confluence.api_token` (Cloud) or `confluence.personal_token` (Server/DC) secret.
2. `mcp-config-set` was used to add the `confluence` block (`url: https://nitinmvs-v2.atlassian.net/wiki`, `username`), preserving the existing `jira` block exactly as it was (verified via the tool's before/after diff output, since there is no direct “read config” tool).
3. `mcp-add` (re)activated the server so its Confluence tools loaded into the session — 26 tools total, e.g. `confluence_search`, `confluence_get_page`, `confluence_create_page`, `confluence_get_page_children`, `confluence_add_comment`.
4. Authentication initially failed (403) because no Confluence-specific API token secret had been set. Once a fresh Atlassian API token was generated (Cloud sites use API tokens, not classic PATs — the `personal_token` schema field is for Server/Data Center only) and stored via `docker mcp secret set atlassian.confluence.api_token=<token>`, `confluence_search` succeeded.

Jira-side tools used throughout: `jira_search` (JQL), `jira_get_issue` (with `include=comments`), `jira_create_issue`, `jira_add_comment`, `jira_get_transitions` / `jira_transition_issue`.

2.2 github-official — pull requests

Used for `create_pull_request` (e.g. PR #11 for TC-9). Fell back to this MCP tool after discovering the `gh` CLI was not installed on the local machine — `git push` still uses local `Git/credential-manager` directly; only PR *creation* goes through this MCP server.

2.3 code-mode

A meta-tool that lets a single JavaScript-like script call multiple MCP servers' tools together in one execution. Available in this project's gateway but not required for the workflows performed so far (Jira/Confluence/GitHub calls were made directly, one tool at a time).

2.4 Secrets

Docker MCP secrets (`docker mcp secret set <key>=<value>`) are how credentials reach these servers. They are **write-only from Claude's perspective** — there is no tool to read a secret's value back out, only to set or overwrite it. Two tokens were pasted directly into the chat during this project (a Confluence API token and a SonarQube token) — both should be rotated, since a chat transcript is not a secure place for a live credential to persist.

3. Skills — reusable, invokable procedures

`.claude/skills/sonar-scan/SKILL.md` formalizes step 6 of the `CLAUDE.md` workflow (the Sonar scan) into a named, reusable skill:

- **Trigger:** invoked explicitly as `/sonar-scan`, or auto-triggered whenever the workflow calls for a Sonar step.
- **Prerequisites:** local SonarQube running at `localhost:9000`; a valid token read from a `SONAR_TOKEN` environment variable — never hardcoded into a command or file.
- **Procedure:** `mvn clean install → mvn sonar:sonar -Dsonar.host.url=... -Dsonar.token=$SONAR_TOKEN` → read back issues (preferring a Sonar MCP tool if one is ever connected, otherwise the CLI/dashboard output) → triage Bugs/Vulnerabilities (must-fix) vs. Code Smells (fix unless a clear false positive) → fix directly in source, no suppression shortcuts → re-run to confirm clean.
- Explicitly documents that `**/Main.java` is intentionally excluded from coverage requirements via `sonar.coverage.exclusions` in `pom.xml`, so that exclusion should never be “fixed away.”

Skills live under `.claude/skills/<name>/SKILL.md` — a YAML frontmatter block (name, description) followed by plain-markdown instructions. The description is what lets Claude decide when to trigger it automatically.

4. Scheduled automation (cron)

To satisfy the “always update Confluence once a Jira ticket is Done” rule without requiring a manual prompt every time, a recurring job was created with CronCreate.

- **Current job:** ID 28d0d292, cron 17 * * * * (hourly, at :17 past — an intentionally off-round minute to avoid clustering with every other job that defaults to :00).
- **What it does each firing:** runs `jira_search` for `project = TC AND status = Done`, lists the Confluence “TC Board JIRA” folder’s child pages (`confluence_get_page_children`), and for any Done ticket without a matching page (matched by ticket key prefix in the page title), creates one via `confluence_create_page` using `jira_get_issue(include=comments)` to source the description/implementation summary/PR link. It stays silent when there’s nothing new to do.
- **Storage:** persisted to `.claude/scheduled_tasks.json` (`durable: true`), so it survives Claude Code restarts.
- **Locking:** `.claude/scheduled_tasks.lock` is a short-lived marker (`sessionId/pid/acquiredAt`) preventing two processes from writing `scheduled_tasks.json` at the same instant — purely internal bookkeeping, never edited directly.
- **Limitations worth knowing:**
 - Not a server-side webhook — it only fires while Claude Code is open and idle on this machine. If the app is closed, nothing runs until it’s reopened.
 - Recurring jobs **auto-expire after 7 days** regardless of frequency and must be recreated.
 - Frequency was originally set to every 5 minutes at the user’s request, then dialed back to hourly after several consecutive no-op firings (Jira ticket status doesn’t change often enough to justify 12x/hour polling cost).
 - Cancel any job with `CronDelete <job-id>`; list all jobs with `CronList`.

5. End-to-end example: how it all connected for TC-9

TC-9 (“Fix inconsistent error handling in `EmployeeService.updateLastName`”) is the clearest illustration of every piece above working together:

1. **Jira:** `jira_create_issue` (atlassian MCP) created TC-9 in project TC.
2. **Workflow start:** `jira_transition_issue` moved it to In Progress.
3. **Git:** verified the current branch was fully merged with `origin/main` (no diff) before branching, to branch safely without needing an explicit main checkout despite uncommitted, unrelated files sitting in the working tree.
4. **TDD:** the test was changed first to assert the specific exception type, confirmed failing against the unfixed code via `mvn test` (using the IntelliJ- bundled Maven

binary and a Corretto 21 JDK, since no global `mvn/JAVA_HOME` was configured on the machine), *then* the implementation was fixed.

5. **Sonar:** `mvn sonar:sonar` run with a `SONAR_TOKEN` env var (per the sonar-scan skill) — quality gate returned OK, 100% new-code coverage.
6. **GitHub:** `git push`, then `github-official__create_pull_request` opened PR #11 (since the gh CLI wasn't available locally).
7. **README:** updated to document the new 404 behavior, committed and pushed separately.
8. **Jira:** `jira_add_comment` with the full change summary and PR link, then `jira_transition_issue` to In Review.
9. **Confluence (manual first, then automated):** once TC-9 was manually moved to Done in the Jira board, the next run of the same check (initially triggered by hand, later by the scheduled job) created its Confluence page under “TC Board JIRA” automatically — no further manual step required.

6. Quick reference

Concern	Where
Project workflow rules	<code>.claude/CLAUDE.md</code>
MCP server registration	<code>.mcp.json</code>
Sonar scan procedure	<code>.claude/skills/sonar-scan/SKILL.md</code>
Scheduled Jira→Confluence sync	<code>.claude/scheduled_tasks.json</code> (job 28d0d292, hourly)
Scheduler write-lock (internal)	<code>.claude/scheduled_tasks.lock</code>
Check MCP server health	<code>claude mcp list</code>
List/cancel scheduled jobs	<code>CronList / CronDelete <id></code>